Although simple lambdas like this are powerful and can simplify code quite a lot, the real power of lambdas comes when they capture local variables.

A.5.1. Lambda functions that reference local variables

Lambda functions with a *lambda introducer* of `[]` can't reference any local variables from the containing scope; they can only use global variables and anything passed in as a parameter. If you wish to access a local variable, you need to *capture* it. The simplest way to do this is to capture the entire set of variables within the local scope by using a lambda introducer of `[=]`. That's all there is to it—your lambda can now access *copies* of the local variables at the time the lambda was created.

To see this in action, consider the following simple function:

```
std::function<int(int)> make_offseter(int offset)
{
   return [=](int j){return offset+j;};
}
```

Every call to `make_offseter` returns a new lambda function object through the `std::function<>` function wrapper. This returned function adds the supplied offset to any parameter supplied. For example,

```
int main()
{
    std::function<int(int)> offset_42=make_offseter(42);
    std::function<int(int)> offset_123=make_offseter(123);
    std::cout<<offset_42(12)<<","<<offset_123(12)<<std::endl;
    std::cout<<offset_42(12)<<","<<offset_123(12)<<std::endl;
}
```

will write out `54,135` twice because the function returned from the first call to `make_offseter` always adds 42 to the supplied argument, whereas the function returned from the second call to `make_offseter` always adds 123 to the supplied argument.

This is the safest form of local variable capture; everything is copied, so you can return the lambda and call it outside the scope of the original function. It's not the only choice though; you can choose to capture everything by